KIT/BSQBD/KSQBD - noSQL systémy a Big Data

Navigace
- Titulní stránka
  - Nástěnka
  - Moje kurzy
    - JC/UPCUH (LS, 2025/2026)
    - JC/UPCUH (ZSiLS, 2025/2026)_2
    - JC/UPCUH (ZSiLS, 2025/2026)
    - KIT/BUSE (LS, 2025/2026)
    - KST/BPOG1 (LS, 2025/2026)
    - KIT/BBIS (LS, 2025/2026)
    - KIT/BMTA (ZS, 2025/2026)
    - KST/BCSH2 (ZS, 2025/2026)
    - KST/BMOSI (ZS, 2025/2026)
    - KIT/BSQBD/KSQBD (ZS, 2025/2026)
      - Účastníci
      - Známky
      - Zápočet, Zkouška
        - ZADÁNÍ zápočtového projektu
    - KIT/BDAS2 (ZS, 2025/2026)
    - FES/BOZP poučení (ZS, 2025/2026)
    - KST/BCSH1 (LS, 2024/2025)
    - KST/BRPP2 (LS, 2024/2025)
    - 2025 – OS
    - KAM/BTEGR (LS, 2024/2025)
    - KST/BDATS (ZS, 2024/2025)
    - KST/BOOP (LS, 2024/2025)
    - KST/BPOG1 (LS, 2024/2025)
    - KST/BSPWE (LS, 2024/2025)
    - KIT/BOPE2 (ZS, 2024/2025)_2
    - KIT/BOPE2 (ZS, 2024/2025)
    - 2025 – PS1
    - KAM/BAMAT (ZS, 2024/2025)_KOMPAS
    - KAM/BAMAT (ZS, 2024/2025)
    - KIT/BZALG (ZS, 2024/2025)
    - 2025 – IZ (ZS)
    - JC/IPAIH (LS, 2024/2025)
    - JC/IPAIG (ZS, 2024/2025)
    - JC/IPAIF (LS, 2023/2024)
    - Více...

ZADÁNÍ zápočtového projektu

Semestrální projekt - obecné informace & obhajoba

Obecné informace k semestrální práci

1. **Semestrální práci zpracovává každý student individuálně.**
2. **Semestrální práce se skládá z:**
    1. **Funkčního řešení, které musí být nasazené na přidělený server pro kontrolu semestrální práce.**
    2. **Dokumentace.**
    3. **Povinných příloh k dokumentaci.**
    4. Ostatních nepovinných příloh.
3. **Minimální rozsah dokumentace k semestrální práci není stanoven**, ovšem semestrální práce musí mít zpracované jednotlivé části s dostatečnou hloubkou, **doporučený rozsah** je tak stanoven na **10-30 stran od Úvodu k Závěru.**
    - Do doporučeného rozsahu nejsou započteny strany, kde obrázek, tabulka nebo jakákoliv jiná ilustrace tvoří půl strany, apod. a dále nejsou započteny přílohy k dokumentaci semestrální práce.
4. **Je nutné dodržet povinné části semestrální práce a dále specifika vybraného tématu, nelze zpracovat pouze vybrané části semestrální práce, v tom případě nebude semestrální projekt obhajitelný.**
5. Práce musí být vlastním dílem autora, jinak nebude uznána (kontrola plagiátorství), v práci je nutné uvést veškeré použité zdroje a nástroje.

Obhajoba

- **Obhajoba semestrální práce je povinná** a koná se dle vypsaných termínů ve STAGu nebo případně po individuální domluvě s vyučující**, práce není uznána pouhým odevzdáním.**
    - Na obhajobu je nutné dostavit se včas a ideálně s mírnou časovou rezervou, jelikož uvítám pokud se ostatní studující zúčastní co největšího počtu prezentací obhajob semestrálních prací.
- **Na obhajobu nevytvářejte žádnou prezentaci ve formátu PPTX, apod.**, budete používat odevzdanou semestrální práci tedy funkční řešení, dokumentaci k semestrální práci včetně příloh, tedy celý ZIP, který jste odeslali.
- **Prezentace bude cca na 15 minut** - předvedení stěžejních částí, zajímavostí ze zadaného tématu, čím je vaše práce výjimečná a co naopak nesplňuje.
- **Při obhajobě je nutné přesně vědět, jak funguje odevzdané řešení, a to z pohledu architektury, datových toků, jednotlivých dotazů apod., vše musí být reprodukovatelné/spustitelné. Před obhajobou si celé řešení na svém zařízení spusťte, ať časově nenarušujete okénko určené pro další/ho prezentující/ho, tzn. na obhajobu si student ideálně přinese vlastní zařízení (notebook), kde bude mít své řešení plně spuštěné a které bude prezentovat, pokud to není možné, pak řešení spustí na počítači v učebně a své řešení předvede na něm.**
- Následují **otázky vyučující cca 5-10 minut**.y

Volba tématu semestrální práce

Student si vybírá z níže uvedených témat:

1. NoSQL databáze klíč-hodnota - Redis
2. NoSQL databáze dokumentová - MongoDB
3. NoSQL databáze sloupcově orientované - Apache Cassandra
4. The Elastic Stack (ELK)

Semestrální projekt - zadání - pokyny

Pro téma 1,2 a 3 platí následující povinné body semestrální práce, které řádně popište v dokumentaci:

- **ÚVOD**
    - Tato kapitola odpovídá na otázky:
        - Jaké téma semestrální práce popisuje?
        - Co vše se čtenář v seminární práci dozví – oblasti, technologie, případové studie, apod?
        - Co není součástí semestrálního projektu, ale vzhledem k tématu by mohlo být?
        - Nezapomeňte uvést verzi databáze, se kterou pracujete, dodržujte princip, že volíte maximálně tři verze zpětně od aktuální (ne major, tzn. např. u MongoDB nesmí být použita starší verze než 8.0.0., u Redis nesmí být starší verze než 8.2, atd.).
        - Nezapomeňte uvést zvolený/é oficiální obrazy z Docker Hub (můžete použít i bitnami obraz) a odůvodnění výběru, např. u Redisu je nutné, abyste měli funkční RedisInsight, u MongoDB je nutné, abyste měli kontejner s Mongo-express (viz cvičení) nebo MongoCompass (web) - obraz z docker hub: haohanyang/compass-web, apod. Není chybou máte-li řešení postavené na vlastním Dockerfile.
- **1. ARCHITEKTURA**
    - Tato kapitola podrobně popisuje, jak jste provedli nasazení NoSQL databáze, proveďte a uveďte, jak konkrétně jste vybranou databázi nasadili a nastavili v rámci clusteru a jak ji využíváte, tato kapitola musí obsahovat následující podkapitoly:
    - 1.1. Schéma a popis architektury.
        - Vytvořte schéma architektury a vložte jako obrázek např. pomocí draw.io.
        - Podrobně architekturu popište, je nutné minimálně odpověď na tyto otázky:
            - Jak vypadá architektura Vašeho řešení a proč?
            - Jak se případně liší od doporučeného používání a proč?
    - 1.2.Specifika konfigurace
    - Podrobně popište specifikaci Vaší konfigurace. Tato kapitola musí obsahovat následující podkapitoly:
        - 1.2.1. CAP teorém
            - Uveďte, jaké garance Brewerova CAP teorému splňuje Vaše řešení?
            - Uveďte, proč právě tyto garance jsou pro Vaše řešení dostačující?
            - Uveďte řádný popis.
        - 1.2.2. Cluster
            - Minimálně 1.
            - Uveďte kolik clusterů používáte a proč?
            - Uveďte řádný popis.
        - 1.2.3. Uzly
            - Minimálně 3.
            - Uveďte kolik nodů používáte a proč?
            - Uveďte řádný popis.
        - 1.2.4. Sharding/Partitioning
            - Minimálně 3.
            - Uveďte, jak používáte sharding/partitioning?
            - Uveďte, zda používáte pro své řešení indexy a např. i sekundární indexy a proč je považujete za dostačující vzhledem k použitým datům (objem, typ dotazů, rozložení zátěže).
            - Uveďte řádný popis.
        - 1.2.5. Replikace
            - Minimálně 3.
            - Uveďte kolik replikací používáte a proč je považujete za dostačující vzhledem k použitým datům a požadované dostupnosti?
            - Uveďte řádný popis.
        - 1.2.6. Perzistence dat
            - Minimálně 3.
            - Uveďte, jakým způsobem řeší Vaše databáze perzistenci dat?
            - Uveďte, jak pracujte s primární i sekundární pamětí.
            - Uveďte, jak načítáte a ukládáte data.
            - Uveďte řádný popis.
        - 1.2.7. Distribuce dat
            - Z předešlých kapitol vše shrňte a uveďte, jak se data rozdělují, jak je replikujete, jak konkrétně u Vašeho řešení probíhá celková distribuce dat pro zápis/čtení.
            - Uveďte řádný popis - textový popis + screeny + popis uvádějící například skript, který provádí automatické rozdělení dat (pozor na příliš velkou výchozí velikost chunk), počty záznam na jednotlivých uzlech (count),...
        - 1.2.8. Zapezpečení
            - Uveďte, jakým způsobem jste vyřešili zabezpečení databáze a proč?
            - Minimálně je požadována autentizace a autorizace.
            - Upozornění: V případě MongoDB je nutné mít keyfile (ne fixní).
- **2. FUNKČNÍ ŘEŠENÍ**
    - Tato kapitola obsahuje popis návod na zprovoznění funkčního řešení a popis jeho struktury.
    - 2.1. Struktura
        - Popište adresářovou strukturu Vašeho řešení a jednotlivé soubory, docker-compose.yml popište důkladně samostatně v kapitole 2.1.1.
        - 2.1.1. docker-compose.yml
            - Uveďte řádný popis vytvořeného docker-compose.yml - jaké služby spouští, jaké porty, volumes, proměnné prostředí, závislosti a inicializační skripty, atd.
    - 2.2. Instalace
        - Podrobně popište, jak zprovoznit Vaše řešení.
        - Řešení je nutné vytvořit tak, aby využívalo docker a **spuštění probíhalo maximálně automatizovaně pomocí docker-compose.yml, tzn. že docker-compose.yml odkazuje na veškeré skripty, se kterými pracuje a pro zprovoznění není nutné provádět manuální spuštění pomocných skriptů.**
        - **V rámci docker-compose.yml využijte automatické spuštění skriptů poté co se vám spustí kontejnery viz například https://www.baeldung.com/ops/docker-compose-run-script-on-start**
- **3. PŘÍPADY UŽITÍ A PŘÍPADOVÉ STUDIE**
    - Popište pro jaké účely (případy užití) ja daná NoSQL databáze vhodná.
    - Uveďte, pro jaký účel (případ užití) jste si danou databázi zvolili a proč? K čemu Vaše řešení slouží? O jaký případ užití se jedná?
    - Uveďte, proč jste nezvolili jinou NoSQL databázi vzhledem k účelu?
    - Vyhledejte a popište 3 případové studie spojené s vybranou NoSQL databázi.
    - Rozsah každé případové studie musí být alespoň 1/2 A4.
- **4. VÝHODY A NEVÝHODY**
    - Popište, jaké výhody a nevýhody má daná NoSQL databáze.
    - Uveďte, jaké výhody a nevýhody má Vaše řešení a proč?
- **5. DALŠÍ SPECIFIKA**
    - Popis specifických vlastností řešení, pokud nejsou použity žádná specifika, pak uveďte, že vaše řešení je použito jak je doporučeno a nemá vlastní specifika (nic mu nechybí a ani mu nic nepřebývá).
- **6. DATA**
    - Použijte libovolné **3 propojitelné datové soubory (ideálně z jedné kolekce), kdy jeden soubor obsahuje alespoň 5 tis. záznamů, doporučená šíře datového souboru je kolem 10-ti sloupců, např. viz https://www.kaggle.com/datasets/samyakrajbayar/stream platform-comprehensive-dataset?select=README.md**
    - Popis dat bude ve velké míře **zpracován pomocí knihoven jazyka Python** a dále bude doplněn dovysvětlujícími texty.
    - Jaká je distribuce shardů/slotů/partitions/tokenů a počet chunků pro daná data?
    - Jaká je velikost jednotlivých chunků/partitions?
    - Jaký je replication lag na secondary uzlech v shardech, consistency level (cassanra)?
    - Jaký je počet záznamů a velikost na shardu/uzlu?
    - Uveďte:
        - S jakými typy dat Vaše databáze pracuje, jakého jsou formátu a jak s nimi databáze nakládá?
        - Proč jste nezvolili další možné datové struktury pro Vaši databázi?
        - S kolika daty Vaše databáze bude pracovat? Jakého rozsahu jsou ukázková data?
        - Kolik obsahují prázdných hodnot?
        - Jaké úpravy jste s daty prováděli a proč?
        - Jaký je zdroj dat? Uveďte URL adresu.
        - Pomocí skriptů v Python s využitím knihoven Pandas, Numpy, apod. data popište a proveďte základní analýzu dat (základní statistiky - počty, prázdná pole, suma, průměr, grafické zobrazení, apod.
- **7. DOTAZY**
    - Uveďte a popište **30 skutečně NETRIVIÁLNÍCH, navazujících a vzájemně odlišných dotazů včetně řešení.** Všechny tři datasaty musí popisovat jedno společné téma a dotazy musí z těchto dat vycházet tak, aby bylo zřejmé, že nad daty byla provedena reálná analytická, transformační nebo konfigurační práce, nikoli pouze formální ukázka syntaxe.
    - Každý dotaz musí být navržen tak, aby ukazoval reálnou schopnost zvolené databáze řešit analytický, provozní nebo architektonický problém. Není cílem předvést syntaxi jednotlivých příkazů, ale demonstrovat pochopení datového modelu, indexace, distribuce dat, vazeb mezi datasety a pokročilých možností dotazování. Jednoduché dotazy, prosté CRUD operace a formální obměny stejného dotazu nebudou považovány za splnění požadavku.
        - NETRIVIÁLNÍ dotaz je například dotaz využívající v MongoDB aggregate a zároveň unwind a zároveň group a zároveň sort nebo například aggregate a zároveň lookup a zároveň match a zároveň project nebo aggregate a zároveň unset a zároveň ... u Redisu je nutné použít LUA skripty, u Apache Cassandra vlastní funkce.
            - Za netriviální dotaz v MongoDB bude považován zejména takový dotaz, který využívá agregační pipeline a kombinuje více stupňů, mongoDB oficiálně uvádí agregační pipeline jako hlavní mechanismus pro vícestupňové zpracování dat a $lookup jako operaci pro spojování kolekcí.
            - Za netriviální dotaz v Redisu nebude považováno prosté GET, SET, HGETALL, LRANGE nebo obdobný elementární přístup ke struktuře. U Redis Stack je pro skutečné analytické nebo agregační dotazy určen FT.AGGREGATE, zatímco FT.SEARCH je vhodný spíše pro selekci a projekci. Využijte například FT.SEARCH s více podmínkami, filtrem a řazením nad indexovanými JSON dokumenty; FT.AGGREGATE s LOAD, APPLY, GROUPBY, REDUCE, SORTBY; agregaci s výpočtem odvozeného pole a následným seskupením; práci nad více typy struktur, například JSON + HASH + indexy RediSearch; měření dopadu změny indexu nebo rozložení slotů v clusteru na dotaz.
            - Za netriviální dotaz v Apache Cassandře nebude považováno prosté čtení podle partition key bez dalšího vysvětlení. Netrivialita by měla být spojena s vědomou prací s partition key, clustering columns, tokeny, konzistenční úrovní, materialized views, indexy nebo s porovnáním dotazů nad různými modely tabulek. Využijte např. srovnání dvou modelů tabulek pro tentýž use case; dotaz, který ukáže význam partition key a clustering columns; práce s token(partition_key) a vysvětlení rozložení dat; měření dopadu různých consistency level; dotaz doplněný kontrolou přes nodetool tablestats, nodetool status, DESCRIBE KEYSPACE, případně statistiky partition.
        - Za netriviální dotaz **nebude považován** jednoduchý příkaz typu prosté find, select, GET, FT.SEARCH, jednoduchý match, jednoduché filtrování jedním polem, prostý COUNT, ani pouhá variace téhož dotazu s jinou konstantou.
        - Za netriviální dotaz bude považován pouze takový dotaz, který kombinuje více operací, pracuje s vazbami mezi daty, s agregací, se strukturou dokumentů/záznamů, s indexy, s clusterem nebo s distribucí dat a přináší výsledek, který má interpretační hodnotu.
        - Příkazy řádně okomentujete tzn., že každý příkaz zkopírujete z konzole a u každého příkazu uvedete, jaké je jeho obecné chování a jak konkrétně pracuje s daty ve vašem případě a řeší konkrétní úlohu**.**
        - **Předpoklad je takový, že budete mít příkazy z různých kategorií např.:**
            - **"agregační funkce",**
            - **"konfigurace",**
            - **"nested (embedded) dokumenty",**
            - **"indexy"**
            - **"propojování dat a vazeb mezi datasety"**
            - **"agregační a analytické dotazy"**
            - **"transformace a obohacení dat"**
            - **"strukturovaná nebo vnořená data"**
            - **"indexy a optimalizace"**
            - **"distribuce dat, cluster, replikace a chování při výpadku"**
            - **"validace a kvalita dat a kontrola konzistence"**
            - **"pokročilé fulltextové nebo podobnostní vyhledávání"**
            - **takových kategorií je požadováno alespoň 5, kdy u každé "kategorie" uvedete alespoň 6 příkazů.**
    - Každý dotaz musí vracet nějaká data.
    - **Každý dotaz musí vracet různá data. Nelze, aby stejná data vracelo více dotazů.**
    - Dle zvoleného typu databáze využijte i možnost práce s clusterem, replikačním faktorem a shardingem.
    - Pokuste se například (mimo jiné) nasimulovat výpadek některého z uzlů a popište možnosti řešení.
    - Upozornění: V případě MongoDB je nutné mít validační schéma.
- **ZÁVĚR**
    - V závěru pochvalně i kriticky zhodnoťte Vaši semestrální práci, popište hloubku zpracování. Shrňte k jakým závěrům jste došli, co je možné s Vaším řešením vykonávat, apod.
- **ZDROJE A NÁSTROJE**
    - Uveďte řádně všechny zdroje a nástroje, se kterými jste pracovali, abecedně seřazené, a to včetně použitých nástrojů.
- **PŘÍLOHY DOKUMENTACE**
    - **Data**
    - **složka** pojmenovaná Data, která obsahuje:
        - obsahuje minimálně 3 datasety
        - obsahuje Python skript (JupyterLab)
    - **Dotazy**
    - **složka** pojmenovaná Dotazy, která obsahuje:
        - 1 soubor se všemi dotazy, kdy každý dotaz obsahuje zadání v přirozeném jazyce a řešení v příslušeném jazyce vybrané NoSQL databáze (přesný příkaz nebo sada příkazů v jazyce dané technologie), při obhajobě pak budete vysvětlovat, proč je dotaz netriviální a jaké mechanismy databáze využívá a provedete ukázku výsledku a jeho interpretace, co výsledek znamená v kontextu projektu.
    - **Funkční řešení**
    - **Složka** pojmenovaná Funkční řešení, která obsahuje:
        - docker-compose.yml
        - skripty nutné pro zprovoznění
        - případně další složky a soubory nutné pro zprovoznění řešení

Pro téma 4 (The Elastic Stack – ELK) platí následující povinné body semestrální práce, které řádně popište v dokumentaci:

- **ÚVOD**
    - Tato kapitola odpovídá na otázky:
        - Jaké téma semestrální práce popisuje?
        - Co vše se čtenář v semestrální práci dozví – oblasti, technologie, případové studie, apod.?
        - Co není součástí semestrálního projektu, ale vzhledem k tématu by mohlo být?
        - Nezapomeňte uvést verze jednotlivých komponent ELK stacku (Elasticsearch, Logstash, Kibana), se kterými pracujete. Dodržujte princip, že volíte maximálně tři verze zpětně od aktuální (ne major, tzn. např. u Elasticsearch nesmí být použita starší verze než 8.0.0., u ostatních komponent obdobně).
        - Nezapomeňte uvést zvolený/é obrazy z Docker Hub a odůvodnění výběru, např. oficiální obrazy elasticsearch, logstash, kibana včetně důvodu (podpora security, verze, dostupná dokumentace, apod.).
- **1. ARCHITEKTURA**
-
    - Tato kapitola podrobně popisuje, jak jste provedli nasazení ELK stacku, uveďte, jak konkrétně jste jednotlivé komponenty (Elasticsearch, Logstash, Kibana, případně Beats) nasadili a nastavili v rámci clusteru a jak je využíváte. Tato kapitola musí obsahovat následující podkapitoly:
    - 1.1. Schéma a popis architektury
        - Vytvořte schéma architektury a vložte jej jako obrázek, např. pomocí draw.io.
        - Podrobně architekturu popište, je nutné minimálně odpovědět na tyto otázky:
            - Jak vypadá architektura Vašeho řešení a proč? (např. kolik Elasticsearch nodů, kde běží Logstash, jak je připojena Kibana, odkud přichází logy/data).
            - Jak se případně liší od doporučeného používání a proč? (např. zjednodušení pro účely semestrální práce a proč?).
    - 1.2. Specifika konfigurace
    - Podrobně popište specifikaci Vaší konfigurace. Tato kapitola musí obsahovat následující podkapitoly:
        - 1.2.1. CAP teorém
            - Uveďte, jaké garance Brewerova CAP teorému splňuje Vaše řešení (např. jak se v kontextu Elasticsearch projevuje dostupnost a tolerance rozdělení).
            - Uveďte, proč právě tyto garance jsou pro Vaše řešení dostačující.
            - Uveďte řádný popis.
        - 1.2.2. Cluster
            - Minimálně 1.
            - Uveďte, kolik clusterů Elasticsearch používáte a proč?
            - Uveďte řádný popis.
        - 1.2.3. Uzly
            - Minimálně 3.
            - Uveďte, kolik nodů (uzlů) Elasticsearch používáte, jaké mají role (master/data/ingest) a proč jste zvolili právě takové rozdělení.
            - Uveďte řádný popis.
        - 1.2.4. Sharding/Partitioning
            - Minimálně 3.
            - Uveďte, jak používáte sharding/partitioning?
            - Uveď, zda používáte pro své řešení indexy a např. i sekundární indexy a proč je považujete za dostačující vzhledem k použitým datům (objem, typ dotazů, rozložení zátěže).
            - Uveďte řádný popis.
        - 1.2.5. Replikace
            - Minimálně 3.
            - Uveďte, kolik replik používáte a proč je považujete za dostačující vzhledem k použitým datům a požadované dostupnosti.
            - Uveďte řádný popis.
        - 1.2.6. Perzistence dat
            - Minimálně 3.
            - Uveďte, jakým způsobem řeší Vaše řešení perzistenci dat v Elasticsearch (ukládání indexů na disk, práce s primární a sekundární pamětí).
            - Uveďte, jak probíhá načítání a ukládání dat (indexace, refresh intervaly, segmenty).
            - Uveďte řádný popis.
        - 1.2.7. Distribuce dat
            - Z předešlých kapitol vše shrňte a uveďte, jak se data rozdělují, jak jsou replikována a jak konkrétně u Vašeho řešení probíhá celková distribuce dat pro zápis/čtení (např. jak Logstash/Beats zapisují, jak se dotazy směrují na shardy).
            - Uveďte řádný popis – textový popis + screeny + popis uvádějící například skript nebo konfiguraci, která provádí automatické rozdělení dat (pozor na příliš velkou výchozí velikost shardu/chunku), počty záznamů na jednotlivých uzlech (count), apod.
        - 1.2.8. Zabezpečení
            - Uveďte, jakým způsobem jste vyřešili zabezpečení ELK stacku a proč.
            - Minimálně je požadována autentizace a autorizace (např. použití uživatelských účtů, rolí, zabezpečení komunikace TLS mezi komponentami).
            - Uveďte řádný popis.
- **2. FUNKČNÍ ŘEŠENÍ**
    - Tato kapitola obsahuje popis a návod na zprovoznění funkčního řešení a popis jeho struktury.
    - 2.1. Struktura
        - Popište adresářovou strukturu Vašeho řešení a jednotlivé soubory, docker-compose.yml popište důkladně samostatně v kapitole 2.1.1.
    - 2.1.1. docker-compose.yml
        - Uveďte řádný popis vytvořeného docker-compose.yml – jaké služby spouští (Elasticsearch, Logstash, Kibana, případně Beats), jaké porty, volumes, proměnné prostředí, závislosti a inicializační skripty, atd.
    - 2.2. Instalace
        - Podrobně popište, jak zprovoznit Vaše řešení (kroky od stažení repozitáře po první přihlášení do Kibany a ověření, že data přitékají).
        - Řešení je nutné vytvořit tak, aby využívalo Docker a **spuštění probíhalo maximálně automatizovaně pomocí docker-compose.yml, tzn. že docker-compose.yml odkazuje na veškeré skripty, se kterými pracuje, a pro zprovoznění není nutné provádět manuální spuštění pomocných skriptů.**
        - V rámci **docker-compose.yml** využijte automatické spuštění skriptů poté, co se Vám spustí kontejnery (např. inicializační skripty pro vytvoření indexů, ingest pipelines apod.).
- **3. PŘÍPADY UŽITÍ A PŘÍPADOVÉ STUDIE**
    - Popište, pro jaké účely (případy užití) je ELK stack vhodný – např. centrální logování, monitorování, fulltextové vyhledávání, analýza událostí.
    - Uveďte, pro jaký konkrétní účel (případ užití) jste si ELK stack ve Vašem projektu zvolili a proč. K čemu Vaše řešení slouží? O jaký případ užití se jedná?
    - Uveďte, proč jste pro daný účel nezvolili jinou technologii (např. jinou NoSQL databázi, jiný log management nástroj).
    - Vyhledejte a popište 3 případové studie spojené s využitím ELK stacku nebo jeho jednotlivých komponent (Elasticsearch, Logstash, Kibana) v praxi.
    - **Rozsah každé případové studie musí být alespoň 1/2 A4**.
- **4. VÝHODY A NEVÝHODY**
    - Popište, jaké výhody a nevýhody má ELK stack jako celek z hlediska škálování, výkonu, správy a nároků na infrastrukturu.
    - Uveďte, jaké výhody a nevýhody má Vaše konkrétní řešení (konfigurace, architektura, datový model indexů) a proč.
- **5. DALŠÍ SPECIFIKA**
    - Popis specifických vlastností řešení, pokud jsou použita (např. specifické ingest pipeline, pokročilé analyzéry, ILM politiky, specifické vizualizace).
    - Pokud nejsou použita žádná specifika, pak uveďte, že Vaše řešení je použito tak, jak je doporučeno, a nemá vlastní specifika (nic mu nechybí a ani mu nic nepřebývá).
- **6. DATA**
    - Použijte libovolné 3 datové soubory (např. logové soubory, JSON události), kdy jeden soubor obsahuje alespoň 5 tisíc záznamů.Popis dat bude ve velké míře zpracován pomocí knihoven jazyka Python a dále bude doplněn dovysvětlujícími texty.
    - Jaká je distribuce shardů/chunků? - GET _cat/shards?v
    - Jaká je velikost shardů? - GET _cat/allocation?v
    - Jaký je replication lag? - GET _cat/recovery?v
    - Jaký je počet docs/velikost na shardu? - GET _cat/indices?v + GET index/_stats/docs
    - Uveďte:
        - S jakými typy dat Vaše řešení pracuje (např. logy webového serveru, aplikační logy), jakého jsou formátu a jak s nimi ELK stack nakládá.
        - Proč jste nezvolili jiné možné datové zdroje pro Vaše řešení.
        - S kolika daty Vaše řešení bude pracovat? Jakého rozsahu jsou ukázková data?
        - Kolik obsahují prázdných hodnot / chybějících polí?
        - Jaké úpravy jste s daty prováděli a proč (čištění, transformace, anonymizace)?
        - Jaký je zdroj dat? Uveďte URL adresu, případně popis generátoru dat.
        - Pomocí skriptů v Python s využitím knihoven Pandas, Numpy, apod. data popište a proveďte základní analýzu dat (základní statistiky – počty, prázdná pole, suma, průměr, grafické zobrazení, apod.).
- **7. DOTAZY**
    - Uveďte a popište 30 NETRIVIÁLNÍCH různých navazujících příkladů včetně řešení (všechny tři datasety popisují jedno téma) a podrobného vysvětlení jednotlivých příkazů.
        - NETRIVIÁLNÍ DOTAZ je v tomto kontextu netriviální vyhledávací dotaz nebo analytický dotaz nad Elasticsearch, využívající např. kombinaci bool dotazů, filtrování, agregací (terms, date_histogram, pipeline agregace), fulltextových analyzérů, scripted fields apod.
        - Za netriviální dotaz v Elasticsearch nebude považován prostý match, term, jednoduchý _search nebo jednoduchá agregace nad jedním polem. Netrivialita má spočívat v kombinaci vyhledávání, filtrování, bucket agregací, metric agregací, pipeline agregací, práce s analyzéry, nested poli, aliasy, mappingem a indexovou strategií. Bool dotazy a agregace jsou běžně kombinovány a pipeline agregace slouží pro odvozené analytické výsledky.
        - Příkazy řádně okomentujte, tzn., že každý dotaz (např. JSON DSL v Kibana Dev Tools) zkopírujete a u každého dotazu uvedete, jaké je jeho obecné chování a jak konkrétně pracuje s daty ve Vašem případě a řeší konkrétní úlohu.
        - Předpoklad je takový, že budete mít dotazy z různých kategorií, např.:
            - „práce s daty“ – indexace, update_by_query, delete_by_query, reindex apod.,
            - „agregační funkce“ – agregace typu terms, date_histogram, metrics (avg, sum, max), pipeline agregace,
            - „konfigurace“ – dotazy související s nastavením indexů, analyzérů, mappingů,
            - „nested (embedded) dokumenty“ – dotazy využívající nested typy, případně složité struktury dokumentů,
            - „indexy“ – dotazy ilustrující práci s různými indexy, aliasy, přepínání indexů, apod.
        - Takových kategorií je požadováno alespoň 5, kdy u každé „kategorie“ uvedete alespoň 6 příkazů.
    - Každý dotaz musí vracet nějaká data.
    - **Každý dotaz musí vracet různá data. Nelze, aby stejná data vracelo více dotazů.**
    - Využijte možnosti práce s clusterem, replikací a shardingem (např. ukažte, jak se mění výsledek/latence při výpadku uzlu, jak se chová cluster při změně počtu shardů/replik).
    - Pokuste se například (mimo jiné) nasimulovat výpadek některého z uzlů a popište možnosti řešení (co se stane s dostupností dat, jak se cluster zotaví).
- **ZÁVĚR**
    - V závěru pochvalně i kriticky zhodnoťte Vaši semestrální práci, popište hloubku zpracování. Shrňte, k jakým závěrům jste došli, co je možné s Vaším řešením vykonávat, apod.
- **ZDROJE A NÁSTROJE**
    - Uveďte řádně všechny zdroje a nástroje, se kterými jste pracovali, abecedně seřazené, a to včetně použitých nástrojů (např. oficiální dokumentace Elastic, články, tutoriály, datasety).
- **PŘÍLOHY DOKUMENTACE**
    - **Data**
    - **složka** pojmenovaná Data, která obsahuje:
        - minimálně 3 datasety,
        - Python skript (např. Jupyter notebook), který provádí analýzu dat.
    - **Dotazy**
    - **složka** pojmenovaná Dotazy, která obsahuje:
        - 1 soubor se všemi dotazy, kdy každý dotaz obsahuje zadání v přirozeném jazyce a řešení v příslušném jazyce (DSL) pro Elasticsearch.
    - **Funkční řešení**
    - **složka** pojmenovaná Funkční řešení, která obsahuje:
        - docker-compose.yml,
        - skripty nutné pro zprovoznění (např. inicializační skripty, konfigurace Logstash pipeline),
        - případně další složky a soubory nutné pro zprovoznění řešení (konfigurační soubory, exporty dashboardů z Kibany, apod.).

Naposledy změněno: pondělí, 13. dubna 2026, 11.55

◄ Podmínky zápočtu a zkoušky a hodnocení

Přejít na...

Důvody vedoucí k zamítnutí práce - binární kritéria ►